INFORME DE FIN DE UNIDAD

Datos Generales

Asignatura	Base de datos
Ciclo	3 A
Unidad	1
Resultado de aprendizaje de la unidad	Modela bases de datos de problemas reales, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Nombre del Docente	Edwin René Guamán Quinche
Fecha	08/11/2026
Enlace de Git	https://github.com/eduardo2006soto-dot/Portafolio_Base_De_Datos/blob/main/Proyecto.md
Nombres y apellidos del estudiantes	Eduardo José Soto Cabrera

1. TITULO

Gestión de fiestas y control de venta de entradas – **EventPass**

2. CONTEXTO DEL PROBLEMA

En la organización de fiestas y eventos, como reuniones universitarias o celebraciones, es necesario llevar un control de los eventos programados, los clientes que adquieren entradas, los tipos de entradas disponibles y las ventas realizadas por los promotores. Cuando esta información se registra únicamente en cuadernos, mensajes o archivos separados, puede resultar difícil mantenerla ordenada y consultar los datos.

Entre las principales dificultades se encuentran la repetición de información, la posibilidad de confundir los datos de los compradores, la falta de claridad sobre qué tipo de entrada corresponde a cada evento y la dificultad para identificar quién realizó una venta. Estas situaciones pueden afectar a los organizadores, promotores y clientes, porque complican el seguimiento de las ventas y de las entradas disponibles.

Por esta razón, se propone diseñar EventPass, una base de datos que almacene la información de los eventos, clientes, promotores, tipos de entradas y ventas. La información se organizará en

tablas relacionadas mediante claves primarias y foráneas, con el propósito de facilitar su registro y consulta.

3. OBJETIVOS

3.1. Objetivo General

Diseñar e implementar una base de datos relacional para EventPass, mediante los modelos entidad-relación y relacional y el lenguaje SQL, con el fin de organizar la información de eventos, clientes, promotores, entradas y ventas, y facilitar el control de las ventas y de la disponibilidad de entradas.

3.2. Objetivos Específicos

1. Elaborar el modelo entidad-relación y el modelo relacional de EventPass, identificando entidades, atributos, relaciones y cardinalidades, para representar de forma ordenada la información de eventos, entradas, clientes, promotores y ventas.

2. Implementar el esquema de la base de datos en SQL (creación de tablas e inserción de datos) y formular cinco consultas en álgebra relacional con su traducción a SQL, para registrar y consultar las ventas y la disponibilidad de entradas.

4. MODELOS CONCEPTUALES DE BASES DE DATOS

4.1. Modelo Entidad-Relación (E-R)

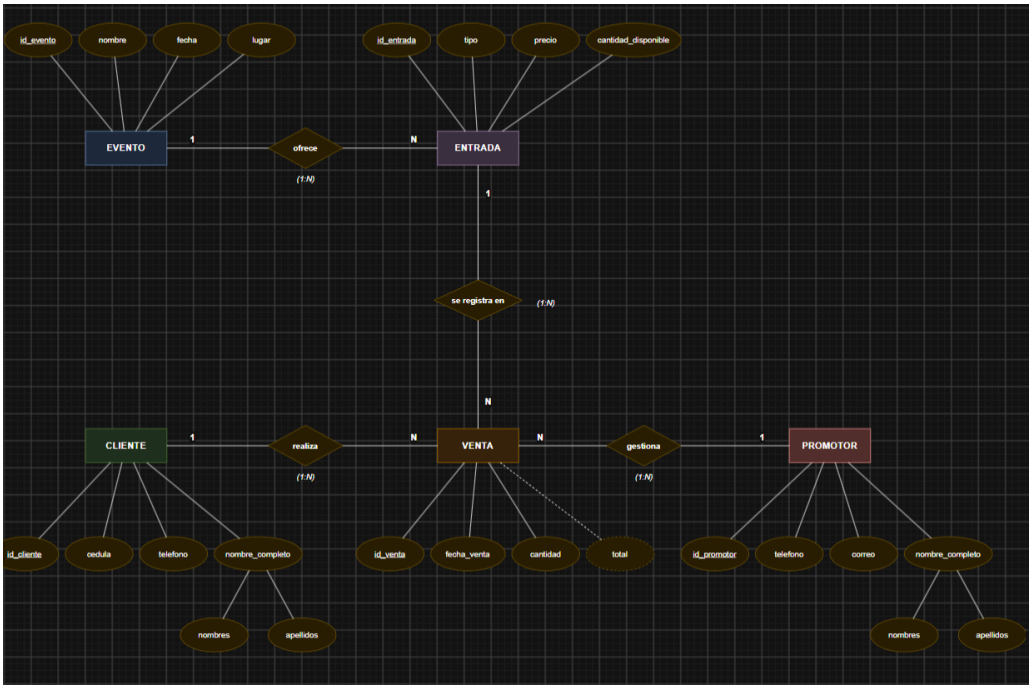


Figura 1. Diagrama Entidad-Relación del sistema EventPass.

4.2. Modelo Relacional

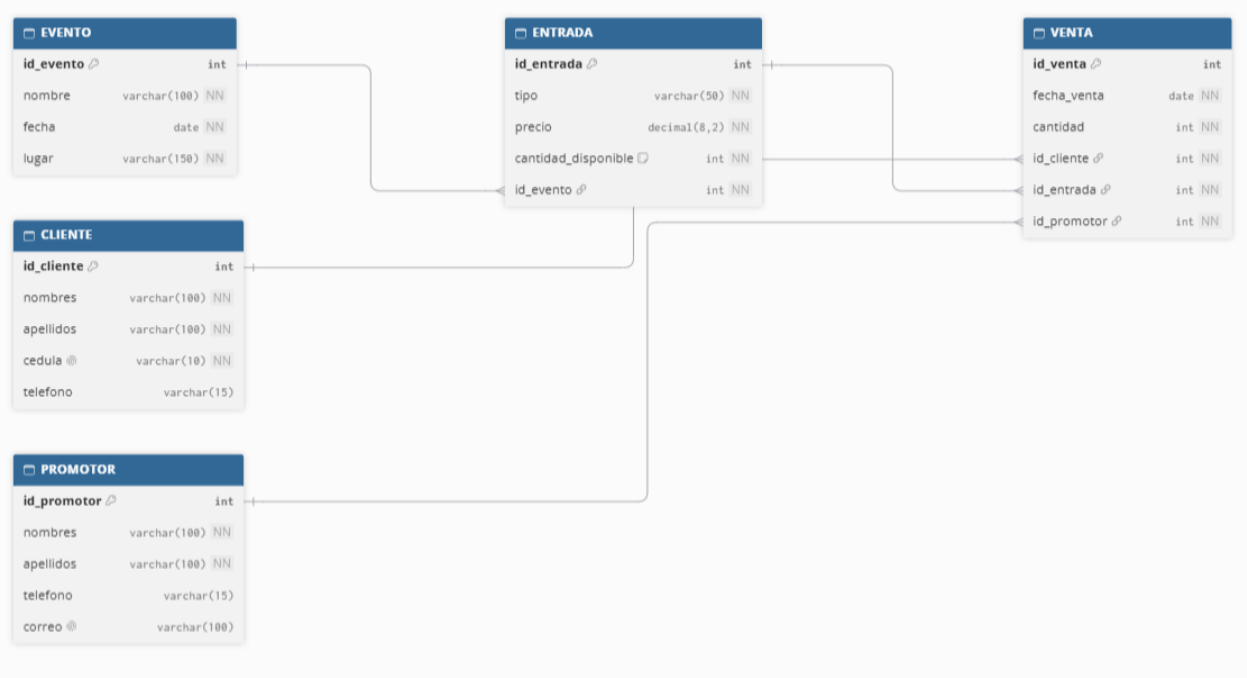


Figura 2. Modelo relacional del sistema EventPass.

PK = clave primaria; FK = clave foránea y la tabla a la que hace referencia.

EVENTO (id_evento PK, nombre, fecha, lugar)

CLIENTE (id_cliente PK, nombres, apellidos, cedula, telefono)

PROMOTOR (id_promotor PK, nombres, apellidos, telefono, correo)

ENTRADA (id_entrada PK, tipo, precio, cantidad_disponible, id_evento FK → EVENTO)

VENTA (id_venta PK, fecha_venta, cantidad, id_cliente FK → CLIENTE, id_entrada FK → ENTRADA, id_promotor FK → PROMOTOR)

CÓDIGO SQL

5.1. Creación de Tablas (DDL)

```sql
CREATE DATABASE IF NOT EXISTS eventpass;
USE eventpass;

CREATE TABLE EVENTO (
    id_evento INT NOT NULL AUTO_INCREMENT,
    nombre    VARCHAR(100) NOT NULL,
    fecha     DATE NOT NULL,
    lugar     VARCHAR(150) NOT NULL,
    PRIMARY KEY (id_evento)
);
```

```sql
CREATE TABLE CLIENTE (
    id_cliente INT NOT NULL AUTO_INCREMENT,
    nombres    VARCHAR(100) NOT NULL,
    apellidos  VARCHAR(100) NOT NULL,
    cedula     VARCHAR(10) NOT NULL,
    telefono   VARCHAR(15),
    PRIMARY KEY (id_cliente),
    UNIQUE (cedula)
);

CREATE TABLE PROMOTOR (
    id_promotor INT NOT NULL AUTO_INCREMENT,
    nombres     VARCHAR(100) NOT NULL,
    apellidos   VARCHAR(100) NOT NULL,
    telefono    VARCHAR(15),
    correo      VARCHAR(100),
    PRIMARY KEY (id_promotor),
    UNIQUE (correo)
);

CREATE TABLE ENTRADA (
    id_entrada          INT NOT NULL AUTO_INCREMENT,
    tipo                VARCHAR(50) NOT NULL,
    precio              DECIMAL(8,2) NOT NULL,
    cantidad_disponible INT NOT NULL DEFAULT 0,
    id_evento           INT NOT NULL,
    PRIMARY KEY (id_entrada),
    UNIQUE (id_evento, tipo),
    CHECK (precio >= 0),
    CHECK (cantidad_disponible >= 0),
    FOREIGN KEY (id_evento) REFERENCES EVENTO(id_evento)
);

CREATE TABLE VENTA (
    id_venta    INT NOT NULL AUTO_INCREMENT,
    fecha_venta DATE NOT NULL,
    cantidad    INT NOT NULL DEFAULT 1,
    id_cliente  INT NOT NULL,
    id_entrada  INT NOT NULL,
    id_promotor INT NOT NULL,
    PRIMARY KEY (id_venta),
    CHECK (cantidad > 0),
```

```
    FOREIGN KEY (id_cliente)  REFERENCES CLIENTE(id_cliente),
    FOREIGN KEY (id_entrada)  REFERENCES ENTRADA(id_entrada),
    FOREIGN KEY (id_promotor) REFERENCES PROMOTOR(id_promotor)
);
```

5.2. Inserción de Datos (DML)

```
INSERT INTO EVENTO
(id_evento, nombre, fecha, lugar) VALUES
(1, 'Fiesta de Bienvenida',   '2026-12-05', 'Salón Central'),
(2, 'Noche de Música en Vivo','2026-12-12', 'Teatro Municipal'),
(3, 'Fiesta de Fin de Año',   '2026-12-31', 'Club Social'),
(4, 'Fiesta Temática 80s',    '2027-01-16', 'Casa de Eventos Norte'),
(5, 'Festival Universitario', '2027-02-20', 'Coliseo Universitario');

INSERT INTO CLIENTE
(id_cliente, nombres, apellidos, cedula, telefono) VALUES
(1, 'Ana María',  'Pérez Loor',    '0900000001', '0991111111'),
(2, 'Carlos',     'Mendoza Ruiz',  '0900000002', '0992222222'),
(3, 'Lucía',      'Vera Castro',   '0900000003', '0993333333'),
(4, 'Diego',      'Salazar Mora',  '0900000004', NULL),
(5, 'Valeria',    'Torres Andrade','0900000005', '0995555555');

INSERT INTO PROMOTOR
(id_promotor, nombres, apellidos, telefono, correo) VALUES
(1, 'Jorge',   'Intriago Paz',  '0981000001', 'jintriago@eventpass.com'),
(2, 'Karen',   'Zambrano Gil',  '0981000002', 'kzambrano@eventpass.com'),
(3, 'Mateo',   'Cedeño Rosado', '0981000003', 'mcedeno@eventpass.com'),
(4, 'Paola',   'Guerrero Vite', NULL,         'pguerrero@eventpass.com'),
(5, 'Esteban', 'Macías Lucas',  '0981000005', NULL);

-- cantidad_disponible = stock inicial - unidades vendidas
INSERT INTO ENTRADA
(id_entrada, tipo, precio, cantidad_disponible, id_evento) VALUES
(1, 'General', 10.00, 194, 1),   -- stock 200, vendidas 6
(2, 'VIP',     25.00,  49, 1),   -- stock 50,  vendidas 1
(3, 'General',  8.00, 141, 2),   -- stock 150, vendidas 9
(4, 'VIP',     20.00,  38, 2),   -- stock 40,  vendidas 2
(5, 'Palco',   35.00,  28, 3),   -- stock 30,  vendidas 2
(6, 'General', 12.00,  95, 3),   -- stock 100, vendidas 5
(7, 'General', 15.00,  80, 4);   -- stock 80,  vendidas 0

INSERT INTO VENTA
```

```
(id_venta, fecha_venta, cantidad, id_cliente, id_entrada, id_promotor) VALUES
(1, '2026-11-01', 2, 1, 1, 1),
(2, '2026-11-02', 1, 2, 2, 2),
(3, '2026-11-03', 4, 3, 1, 1),
(4, '2026-11-03', 3, 4, 3, 3),
(5, '2026-11-04', 2, 5, 5, 2),
(6, '2026-11-05', 5, 1, 6, 4),
(7, '2026-11-06', 2, 2, 4, 3),
(8, '2026-11-07', 6, 3, 3, 5);
```

CONSULTA DE ÁLGEBRA RELACIONAL Y TRADUCIR A SQL

A continuación, se presentan cinco consultas sobre la base de datos EventPass. Cada una se formuló primero en álgebra relacional, se comprobó en la herramienta Relational y luego se tradujo a SQL. Cada consulta utiliza un operador distinto del álgebra relacional.

#	Operador	Qué consulta
1	σ y π	Entradas con precio ≥ 20 y más de 30 unidades disponibles.
2	⋈ (4 tablas)	Ventas con id_venta ≥ 5, con su cliente, evento y tipo de entrada.
3	− (diferencia)	Entradas que nunca se han vendido.
4	∩ (intersección)	Clientes que compraron en la Fiesta de Bienvenida y en la Fiesta de Fin de Año.
5	÷ (división)	Clientes que compraron en todos los eventos con id_evento ≤ 2.

Consulta 1 — Selección y proyección (operadores σ y π)

Descripción del objetivo de la consulta: Obtener las entradas de precio alto (mayor o igual a 20) que todavía tienen buena disponibilidad (más de 30 unidades), mostrando su identificador, precio y cantidad disponible.

Álgebra Relacional:

> π id_entrada, precio, cantidad_disponible (σ precio >=20 and cantidad_disponible > 30 (entrada))

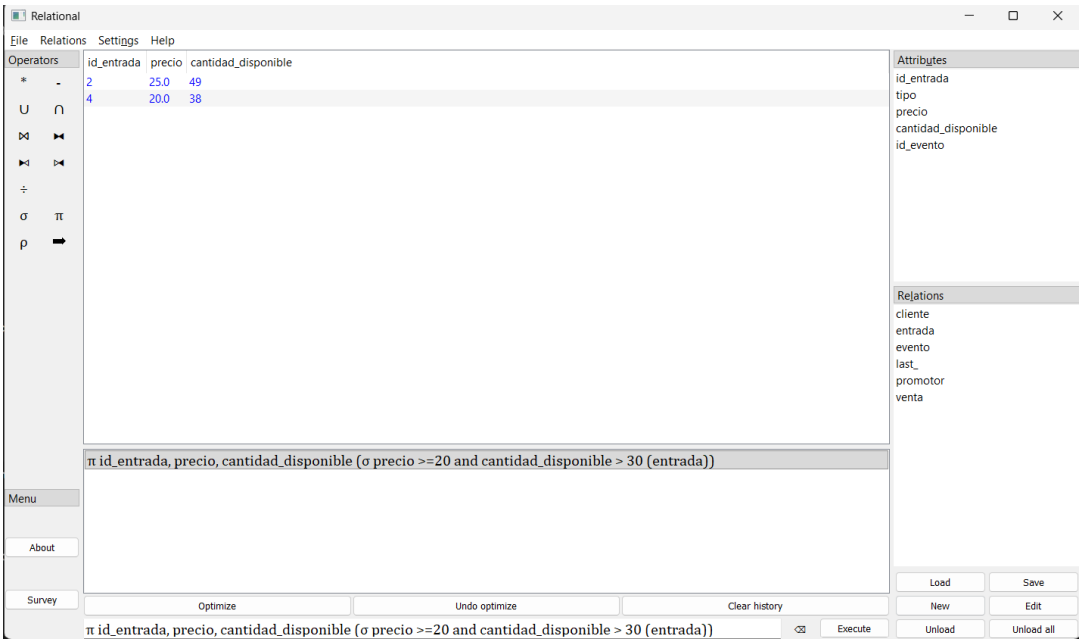


Figura 3. Resultado de la consulta 1 en Relational.

SQL:

```
SELECT id_entrada, precio, cantidad_disponible
FROM   ENTRADA
WHERE  precio >= 20 AND cantidad_disponible > 30;
```

Resultado esperado: Se obtienen 2 registros: la entrada 2 (precio 25.00, 49 disponibles) y la entrada 4 (precio 20.00, 38 disponibles). La entrada 5 (Palco, 35.00) no aparece porque solo tiene 28 disponibles.

Consulta 2 — Reunión natural de cuatro tablas (operador ⋈)

Descripción del objetivo de la consulta: Listar las ventas con número de venta mayor o igual a 5 (registradas del 4 al 7 de noviembre de 2026), indicando quién compró, para qué evento, qué tipo de entrada, cuántas unidades y el precio de la entrada.

Álgebra Relacional:

> π nombres, apellidos, nombre, tipo, cantidad, precio (σ id_venta >= 5 (cliente ⋈ venta ⋈ entrada ⋈ evento))

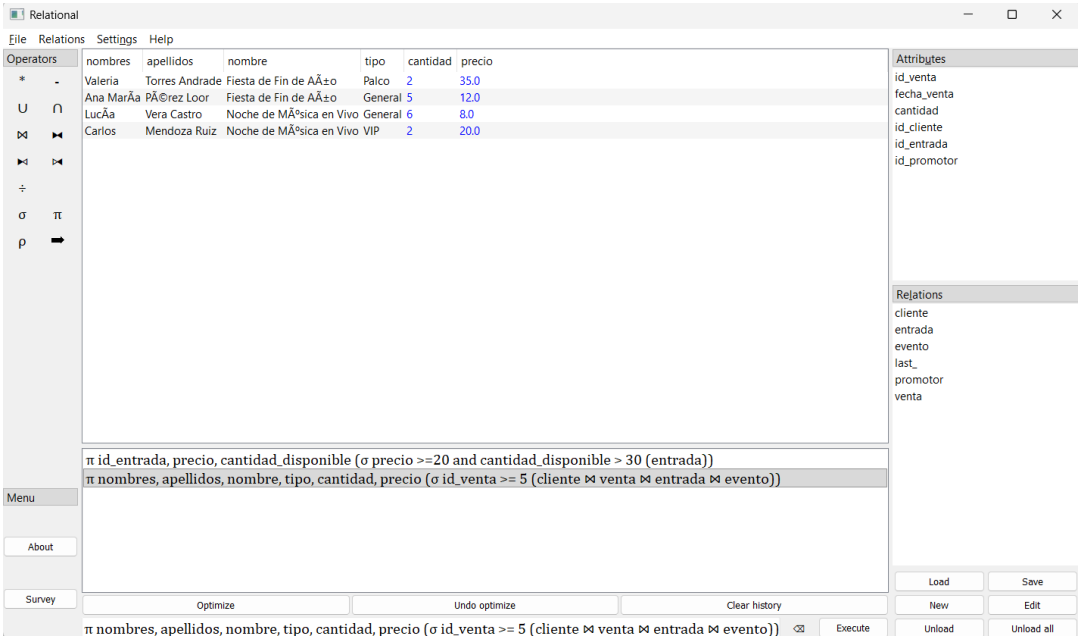


Figura 4. Resultado de la consulta 2 en Relational.

SQL:

```
SELECT c.nombres, c.apellidos, ev.nombre, e.tipo, v.cantidad, e.precio
FROM   CLIENTE c
JOIN   VENTA v   ON v.id_cliente = c.id_cliente
JOIN   ENTRADA e ON e.id_entrada = v.id_entrada
JOIN   EVENTO ev ON ev.id_evento = e.id_evento
WHERE  v.id_venta >= 5;
```

Resultado esperado: Se obtienen 4 registros: Valeria Torres Andrade (Fiesta de Fin de Año, Palco, 2 unidades, 35.00); Ana María Pérez Loor (Fiesta de Fin de Año, General, 5 unidades, 12.00); Lucía Vera Castro (Noche de Música en Vivo, General, 6 unidades, 8.00) y Carlos Mendoza Ruiz (Noche de Música en Vivo, VIP, 2 unidades, 20.00).

Consulta 3 — Diferencia (operador −)

Descripción del objetivo de la consulta: Identificar las entradas que nunca se han vendido, es decir, las que existen en ENTRADA pero no tienen ninguna venta asociada. Sirve para detectar entradas que necesitan más promoción.

Álgebra Relacional:

> π id_entrada, tipo, precio, id_evento (entrada) - π id_entrada, tipo, precio, id_evento (entrada ⋈ venta)

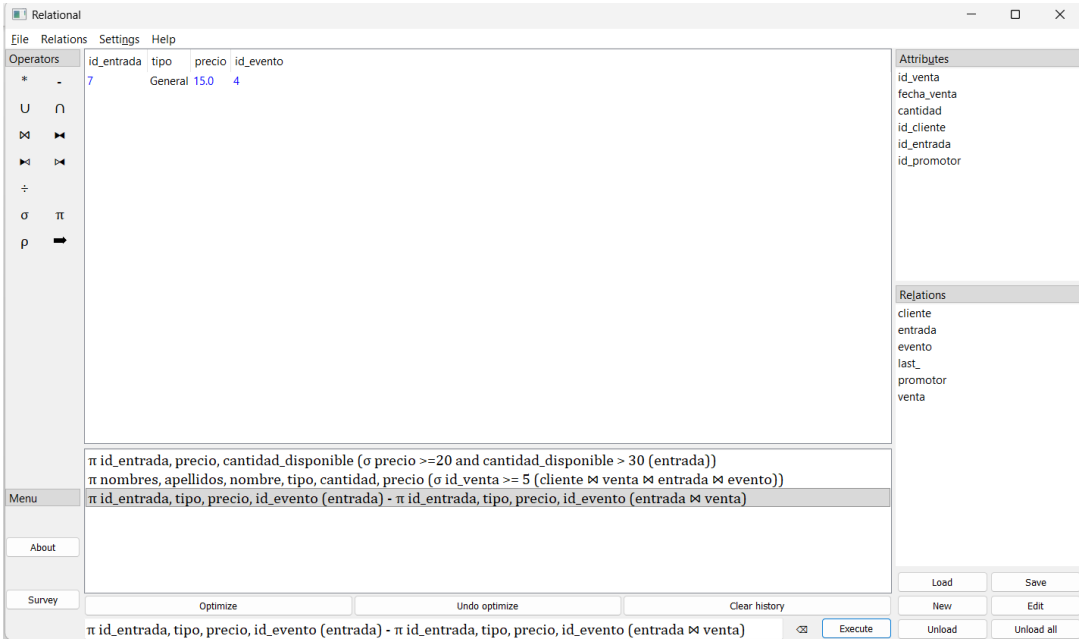


Figura 5. Resultado de la consulta 3 en Relational.

SQL:

```
SELECT id_entrada, tipo, precio, id_evento
FROM   ENTRADA
WHERE  id_entrada NOT IN (SELECT id_entrada FROM VENTA);
```

Resultado esperado: Se obtiene 1 registro: la entrada 7 (General, 15.00) del evento 4 (Fiesta Temática 80s), que todavía no registra ventas.

Consulta 4 — Intersección (operador ∩)

Descripción del objetivo de la consulta: Encontrar los clientes que compraron entradas tanto para la Fiesta de Bienvenida (evento 1) como para la Fiesta de Fin de Año (evento 3). La intersección conserva únicamente los clientes que aparecen en ambos conjuntos.

Álgebra Relacional:

```
e1 = π id_cliente (t1 ⋈ (σ id_evento <= 1 (evento)))
e3 = π id_cliente (t1 ⋈ (σ id_evento >= 3 (evento)))
(e1 ∩ e3) ⋈ cliente
```

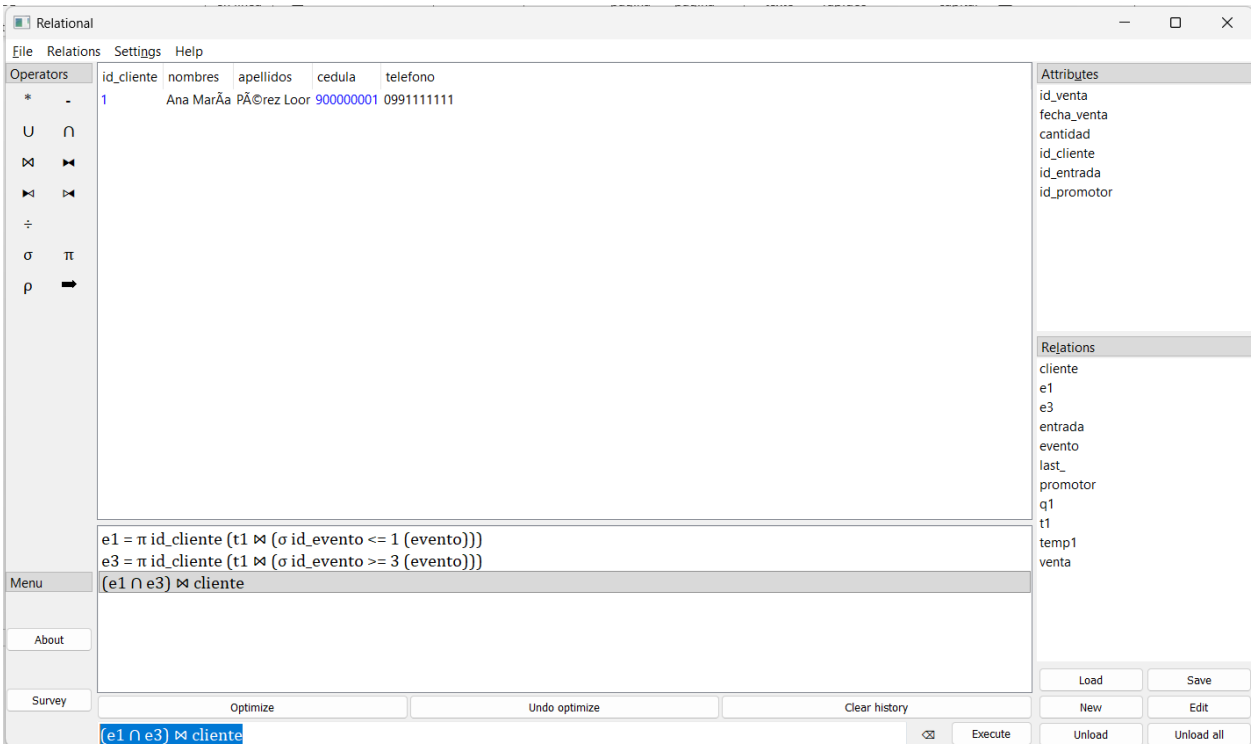


Figura 6. Resultado de la consulta 4 en Relational.

SQL:

```
SELECT c.*
FROM   CLIENTE c
WHERE  c.id_cliente IN (SELECT v.id_cliente
                        FROM   VENTA v
                        JOIN   ENTRADA e ON e.id_entrada = v.id_entrada
                        WHERE  e.id_evento = 1)
  AND  c.id_cliente IN (SELECT v.id_cliente
                        FROM   VENTA v
                        JOIN   ENTRADA e ON e.id_entrada = v.id_entrada
                        WHERE  e.id_evento = 3);
```

Resultado esperado: Se obtiene 1 registro: Ana María Pérez Loor (id_cliente 1), la única persona que compró entradas para ambos eventos.

Consulta 5 — División (operador ÷)

Descripción del objetivo de la consulta: Encontrar los clientes que han comprado entradas en todos los eventos con id_evento menor o igual a 2 (Fiesta de Bienvenida y Noche de Música en Vivo). Es una consulta de división: se buscan los clientes relacionados con todos los elementos de un conjunto. En SQL se implementa agrupando por cliente y comparando cuántos de esos eventos compró con el total de eventos del conjunto.

Álgebra Relacional:

> (π id_cliente, id_evento (venta ⋈ entrada) ÷ π id_evento (σ id_evento <= 2 (entrada))) ⋈ cliente

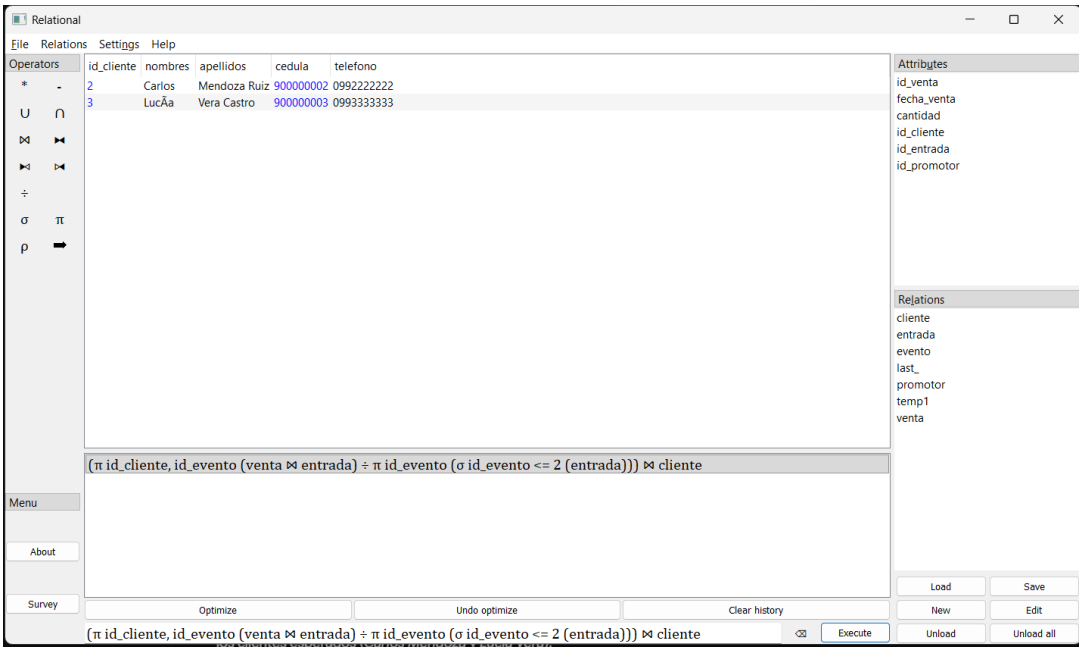


Figura 7. Resultado de la consulta 5 en Relational.

SQL:

```
SELECT   c.id_cliente, c.nombres, c.apellidos, c.cedula, c.telefono
FROM     CLIENTE c
JOIN     VENTA v   ON v.id_cliente = c.id_cliente
JOIN     ENTRADA e ON e.id_entrada = v.id_entrada
WHERE    e.id_evento <= 2
GROUP BY c.id_cliente, c.nombres, c.apellidos, c.cedula, c.telefono
HAVING   COUNT(DISTINCT e.id_evento) =
         (SELECT COUNT(DISTINCT id_evento)
          FROM   ENTRADA
          WHERE  id_evento <= 2);
```

Resultado esperado: Se obtienen 2 registros: Carlos Mendoza Ruiz (id_cliente 2) y Lucía Vera Castro (id_cliente 3), quienes compraron entradas en los dos eventos del conjunto.

CONCLUSIONES

Conclusión 1: Al elaborar el diagrama entidad-relación y el modelo relacional de EventPass, identificar las entidades, los atributos y las relaciones me resultó muy sencillo. La dificultad apareció en el modelo relacional, al crear las nuevas relaciones (tablas) a partir del diagrama, un paso que se me complicó bastante. El aprendizaje que me dejaron ambos modelos es que todo funciona a través de entidades, y que algunas de ellas pueden ser fuertes o débiles, según dependan o no de otra entidad para existir. Comprender esto fue la base para construir correctamente las tablas de la base de datos.

Conclusión 2: La escritura del código SQL (DDL y DML) me ayudó a comprender mejor el tema, cómo funcionan las bases de datos, cuán extensas pueden llegar a ser y cómo, a través del lenguaje SQL, todo ese conjunto funciona. También comprobé que, si el modelo relacional se hace mal, no

es posible construir un modelo correcto en SQL, porque las tablas, las claves y sus relaciones dependen directamente de ese diseño. Lo que más me gustó fue ver cómo se referencian las claves foráneas en SQL mediante FOREIGN KEY … REFERENCES, que mantiene la integridad entre las tablas.

Conclusión 3: El uso del álgebra relacional facilita entender y encontrar de manera inmediata ciertas características dentro de una base de datos, sin necesidad de ir buscando registro por registro. Con la herramienta Relational pude comprobar el resultado de cada consulta antes de traducirla a SQL . Las consultas que me resultaron más útiles fueron la reunión natural (JOIN), la selección y la división, porque las considero las más aprovechables a lo largo de mi vida estudiantil.

BIBLIOGRAFÍA

[1] R. Guamán-Quinche, "Álgebra Relacional," presentación, Carrera de Ingeniería en Sistemas/Computación, Facultad de la Energía, las Industrias y los Recursos Naturales No Renovables, Universidad Nacional de Loja, Loja, Ecuador, 2024.

https://www.canva.com/design/DAGb1TJ2ktA/bYAbb_nMQVgrdUtzz6EZMg/edit